很好，你现在进入了一个非常关键的阶段。

你的状态我先重新定位一下：

当前能力树

```
AI 基础
├── D2L Transformer ✅
├── Karpathy GPT from scratch ✅
├── LLaMA 基础概念 ✅

CUDA
├── CUDA 基础 ✅
├── Memory Hierarchy ✅
├── Vectorization ✅
├── Reduce ✅
├── GEMM ✅
└── Online Softmax ✅

环境
├── RTX 5070 Ti ✅
├── CUDA 环境 ✅
├── PyTorch CUDA ✅
├── Nsight Compute ✅（待深入）
```

你现在已经不是 CUDA 入门阶段，而是进入：

> **AI Infra Kernel Engineer 路线第二阶段：LLM 核心算子 + 性能优化**

结合你之前确定的目标：

- 2027 年 3/4 月开始投 AI Infra 实习
- 方向：AI Infra / 推理优化，不走大模型算法
- 参考 AIInfraGuide + AI Infra Atlas

我建议你后续按照下面路线推进。

总路线（10月～明年3月）

现在
 |
 ↓
LayerNorm / RMSNorm
 |
 ↓
Tensor Core GEMM
 |
 ↓
Nsight Compute 深度分析
 |
 ↓
Kernel Fusion
 |
 ↓
Triton
 |
 ↓
FlashAttention
 |
 ↓
KV Cache
 |
 ↓
PagedAttention
 |
 ↓
Mini-vLLM
 |
 ↓
vLLM源码
 |
 ↓
实习

第一阶段（现在～10月中旬）

LayerNorm + RMSNorm

目标：

> 掌握 Transformer Block 中最常见的 Normalization Kernel

学习资料

1. AIInfraGuide（主线）

继续沿着：

AIInfraGuide

看：

```
CUDA模块
 ↓
算子优化
 ↓
Normalization相关内容
```

2. 论文/原理

推荐：

Root Mean Square Layer Normalization

重点：

不是为了读论文，而是理解：

为什么 LLaMA 放弃 LayerNorm 使用 RMSNorm。

实战任务

不要只看。

直接建立：

```
cuda-kernel-lab
```

结构：

```
cuda-kernel-lab/

├── reduce
├── gemm
├── softmax

├── layernorm
│   ├── layernorm_v0.cu
│   ├── layernorm_v1.cu
│   └── benchmark

└── rmsnorm
    ├── rmsnorm_v0.cu
    ├── rmsnorm_v1.cu
    └── benchmark
```

版本：

LayerNorm V0

Naive:

```
每个block负责一个row

reduce mean

reduce variance

normalize
```

LayerNorm V1

优化：

```
warp shuffle reduce
```

LayerNorm V2

优化：

```
float4 vector load
```

RMSNorm 同样。

最终记录：

```
PyTorch
↓
CUDA naive
↓
CUDA optimized
```

性能：

```
Latency
Bandwidth
Speedup
```

第二阶段（10月中旬～10月底）

Tensor Core + Nsight Compute

这是你5070 Ti真正发挥作用的地方。

你的5070 Ti：

- Blackwell架构
- 支持Tensor Core
- 适合做FP16/BF16 Kernel

学习资料

CUDA官方

NVIDIA CUDA C++ Programming Guide

重点：

```
Tensor Core Programming
WMMA
mma
```

CUTLASS

NVIDIA CUTLASS GitHub

不要学习全部。

只看：

```
GEMM example

threadblock tile

warp tile

pipeline
```

实战

你的GEMM升级：

现在：

```
CUDA Core GEMM
```

升级：

```
Tensor Core GEMM
```

目标：

理解：

```
Global Memory

↓

Shared Memory

↓

Register Fragment

↓

Tensor Core MMA

↓

Accumulator
```

最后比较：

```
你的GEMM

vs

cuBLAS
```

第三阶段（10月底～11月）

Kernel Fusion

这是AI Infra非常重要的能力。

资料：

继续看：

AIInfraGuide

重点：

```
Fusion
Memory Optimization
```

做三个Fusion：

1. Residual + RMSNorm

普通：

```
add kernel
↓
RMSNorm kernel
```

优化：

```
一个kernel完成
```

2. RMSNorm + RoPE

这是LLM里面真实结构。

3. SwiGLU

LLaMA MLP：

```
gate projection
↓
SiLU
↓
multiply
↓
down projection
```

第四阶段（11月）

Triton

这时候再学。

不要提前。

因为你现在已经懂：

- thread
- warp
- memory
- reduce
- tiling

学习资料：

官方教程

OpenAI Triton GitHub

重点：

```
tutorials
```

练习：

顺序：

```
vector add

↓

softmax

↓

layernorm

↓

matmul

↓

attention
```

推荐：

Triton Puzzles GitHub

这个非常适合你。

第五阶段（11月底～12月）

FlashAttention

你现在的基础已经够。

因为：

FlashAttention：

本质：

```
GEMM
+
Softmax
+
Tiling
+
Online Softmax
+
Memory Optimization
```

你已经会：

✅ GEMM

✅ Online Softmax

资料：

论文：

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

源码：

FlashAttention GitHub

学习顺序：

不要直接看源码。

先：

FlashAttention为什么快？

↓

IO-aware

↓

tiling

↓

online softmax

↓

CUDA实现

第六阶段（12月～1月）

LLM Inference

进入 AI Infra 真正核心。

学习：

KV Cache

理解：

Prefill:

```
prompt
 ↓
QKV
 ↓
保存 KV
```

Decode:

```
新token

↓

读取历史KV

↓

生成
```

资料：

推荐：

vLLM GitHub

先不要读全部源码。

看：

```
engine

scheduler

attention backend

kv cache
```

第七阶段（1月～2月）

Mini-vLLM项目

这是你的第二个核心简历项目。

目标：

自己实现：

```
request queue

↓

scheduler

↓

continuous batching

↓

KV cache

↓

paged attention

↓

generation
```

你的简历项目规划

到投实习：

应该有三个东西：

项目1

CUDA Kernel Library

包含：

```
Reduce

GEMM

Softmax

LayerNorm

RMSNorm

FlashAttention
```

项目2

Mini-vLLM

包含：

```
KV Cache

PagedAttention

Continuous batching
```

项目3

开源贡献

目标：

至少：

```
vLLM issue

FlashInfer issue

Triton PR
```

你现在这一周具体任务

因为你刚装好5070Ti，我建议：

Day1

环境确认：

```
nvidia-smi

nvcc

torch.cuda

ncu
```

跑：

- Reduce benchmark
- GEMM benchmark
- Softmax benchmark

记录5070Ti数据。

Day2-Day4

LayerNorm：

完成：

- PyTorch baseline
- CUDA V0
- CUDA V1 warp reduce

Day5-Day6

RMSNorm：

完成：

- CUDA V0
- CUDA V1
- benchmark

Day7

整理：

GitHub：

```
cuda-kernel-lab
```

写：

README：

```
算法

优化点

性能数据

NCU分析
```

你现在最大的优势是：

很多 AI Infra 初学者：

```
会PyTorch
不会CUDA
```

而你已经：

```
CUDA kernel
+
GEMM优化
+
Reduce吧0
+
Online Softmax
```

所以接下来不要再花大量时间学模型结构。

你的主线应该变成：

> **用 CUDA 能力解决 LLM 推理中的真实性能问题。**

你的 RTX 5070 Ti 从现在开始，主要用途不是训练模型，而是：

- 跑 Kernel benchmark
- 做 NCU profiling
- 调 FlashAttention
- 跑 Mini-vLLM
- 做推理优化实验

这正好匹配你的 AI Infra 目标。